Fix the Program, Not the Schedule

Bitwise-Reproducible LLM Serving with Every Optimization On

Haocheng Lu

November 2026

Abstract

The same request sent to an LLM serving engine returns different tokens and log-probabilities depending on what else is in the batch, whether its prefix is cached, where chunked prefill cuts the prompt, and whether a token was produced by decode or by prefill. An RL trainer that rescores the rollout gets yet another value. Deterministic modes in today's engines pin or disable the optimizations responsible, pay for it in throughput, and still leave prefill and decode numerically different.

We fix the *program* instead of the schedule. Every query row and every output element executes one canonical sequence of floating-point operations that depends only on positions and shapes, and the sequence is chosen so that its independent parts are exactly the partitions fast kernels want. For attention it is an online softmax over 32-key tiles inside absolute 512-key fold groups, folded in a fixed order: prefill keeps the FlashAttention-2 structure, decode parallelises over groups, and prefill, chunked prefill, prefix-cache hits and decode become bitwise identical by construction. For projections each output element runs one fixed MMA chain cut into a number of segments set by the weight's shape, which leaves tiling, pipelining, stream-K and segment placement free for an autotuner; declaring the largest tensor-parallel degree in that rule makes the output independent of the degree as well.

In a Rust serving engine with radix caching, chunked prefill, CUDA graphs and CPU/GPU overlap all enabled, `--deterministic` is bitwise reproducible in tokens and log-probabilities across every load, cache state and configuration we test. A PyTorch forward on the same kernels, run the way an RL trainer packs micro-batches, reproduces the engine's log-probability for all 55,593 sampled tokens on Qwen3 0.6B–8B; a standard bf16 trainer forward is off by more than 1% in importance ratio on 10–22% of them. On an A800 the deterministic engine runs at −5.9% to +14.2% of its own non-deterministic throughput across 40 workloads, matches vLLM 0.17.1 within 2.8% where it trails, and delivers 2.0–3.7× the throughput of vLLM 0.17.1's batch-invariant mode, the only other engine that passes our numeric checks. Kernel outputs and GSM8K generations are bitwise identical between an A800 and an RTX 4090 D. The same principle lets Gated DeltaNet decode run the trainer's own chunked program inside vLLM, bitwise, with speculative decoding on or off, at 0.96–1.08× stock throughput.

1 Introduction

Reproducible inference underpins evaluation, debugging, auditing and, increasingly, reinforcement learning, where the rollout engine's log-probabilities enter the trainer's importance ratios. Yet greedy decoding of the same request is not reproducible in production engines. Floating-point addition is not associative, and a serving engine changes the order of its reductions every time it changes how work is partitioned:

- the batch size changes GEMM tiling and split-K;
- the load changes how many KV splits decode attention uses;
- chunked prefill moves where a prompt's attention is cut;
- a prefix-cache hit turns part of a prefill into a read of cached KV;
- a token produced by the decode kernel is recomputed by the prefill kernel when the conversation continues, a request is preempted, or a trainer scores the rollout.

The established remedy is *batch-invariant* kernels [1]: one reduction order regardless of batch size, obtained by fixing the KV split size or not splitting at all and by pinning GEMM configurations. This removes batch dependence at a price. The optimizations that adapt to load are exactly the ones removed, so throughput drops — SGLang reports 34% on H100 [2], and we measure vLLM's batch-invariant mode at 30–49% of its own throughput. And prefill still disagrees with decode: the prefill kernel rounds probabilities against a running maximum over the whole history, the decode kernel against the maximum of its split.

Determinism is a property of a program, not of a schedule. Fix, for each query row and each output element, one canonical sequence of floating-point operations that depends only on positions and shapes. Any kernel that executes exactly that sequence produces the same bits, however rows are batched and whichever kernel runs them. The sequence still leaves the schedule free wherever it has independent parts, and a kernel may split work at those points and only there. The choice of canonical sequence therefore decides both *which* partitions are free and *how fast* the deterministic kernels can be. Serving determinism stops being a constraint imposed on kernels and becomes a

design problem: pick the canonical sequence whose free partitions are the ones fast kernels already use.

Attention. Fast prefill kernels (FlashAttention-2 [3], FlashInfer [4]) stream a query tile over key tiles with an online softmax; fast decode kernels split long contexts over CTAs and merge partial softmax states. We cut keys at *absolute* positions into fold groups of 512 keys. Within a group the canonical sequence is an online softmax over 32-key tiles in ascending order from the empty state; across groups it is a fixed-form ascending fold (Figure 1). A prefill kernel is then a FlashAttention-2 kernel that restarts its running state every 512 keys, a decode kernel gives each group its own CTA, and chunked prefill, prefix-cache hits and re-prefill of a decoded token all execute the identical per-row sequence.

Projections. A row of $XW^\top$ is reproducible if every output element accumulates K in one fixed order. Tile sizes along M and N never enter that order, so the GEMM can use small balanced tiles for decode and large tiles for prefill, chosen by an autotuner that cannot change results (Figure 2). Where decode needs split-K, the canonical program supplies it: a fixed number of K segments per weight shape, folded in a fixed order.

Contributions.

1. **Formulation (§3).** Serving determinism as a canonical per-row program with free partitions at its associativity points, with invariance lemmas for attention, GEMM and their composition, and an explicit numerical contract: which parameters define the bits, and which a kernel may choose.
2. **PI-Attn kernels (§4).** Ampere/Ada CUDA kernels for paged prefill, extend, decode and merge that share one canonical sequence; and a row-invariant GEMM family — small-M streaming kernels, CUTLASS tiles, deterministic stream-K that hands the fp32 accumulator from worker to worker, and segment-parallel kernels — all bitwise identical to each other.
3. **A deterministic serving engine (§5).** `--deterministic` in a Rust/libtorch engine with radix prefix caching, chunked prefill, mixed batches, CUDA graphs and CPU/GPU overlap, all left on, returning bitwise-reproducible tokens and log-probabilities.
4. **Rollout equals trainer (§6.3).** A PyTorch forward on the engine's kernels reproduces every sampled token's log-probability bit for bit, whatever micro-batch packing the trainer uses: importance ratios are exactly 1.
5. **Exact Gated DeltaNet decode (§7).** The same principle for linear attention: hybrid models decode with the trainer's own chunked program, with speculative verification and rollback for free, bitwise inside vLLM at stock throughput.
6. **Tensor-parallel degree as a canonical parameter (§8).** Column-parallel projections plus a declared $t_{\max}$ in the segment rule make outputs identical for every degree.
7. **Evaluation (§6).** *detcheck*, a black-box checker for any OpenAI-compatible server that compares tokens and log-probabilities bit for bit, and a test suite validated against deliberately broken kernels.

2 Background and related work

2.1 Where serving nondeterminism comes from

Table 1. Serving optimizations and the reductions they reorder.

optimization	reduction whose order changes	usual deterministic-mode remedy
continuous batching	GEMM tile and split-K follow M	pin one GEMM config, no split-K
split-KV decode	number of KV splits follows batch and length	fixed split size, or one split
chunked prefill	where a prompt's attention is cut	align chunks to the split size
prefix cache	cached prefix read by a different kernel or tiling	restrict or disable the cache
prefill vs decode	token p decoded by one kernel, re-processed by another	none
CUDA-graph padding, page size, batch limit	shapes seen by kernels	pin them
sampling	random stream indexed by batch row or request start	per-request generator

The prefill/decode row is the subtle one. An online-softmax prefill kernel rounds each probability $P = \mathrm{bf16}(e^{s-m_{\mathrm{run}}})$ against a running maximum that depends on every earlier key tile of the query; a decode kernel with a fixed split size

rounds against the maximum of its split. The two produce different bf16 values of P for the same score, so fixing the split size cannot make them agree.

2.2 Related work

Batch-invariant kernels. Thinking Machines [1] identified batch-size dependence as the dominant source of serving nondeterminism and proposed batch-invariant RMSNorm, matmul, and attention with a fixed decode split size. SGLang's deterministic inference [2] builds on it with fixed split sizes, chunked-prefill alignment and a dedicated sampler; on our GPU its FlashInfer path also disables the radix cache. vLLM's `VLLM_BATCH_INVARIANT` overrides aten matmul and reductions with batch-invariant Triton kernels and runs attention unsplit; it targets compute capability 9.0 and up, yet 0.17.1 is deterministic on our A800 (§6.1). An open vLLM pull request for DeepSeek-V4.1-Flash [5] routes every causal row through the unsplit sparse decode kernel so that a token decodes to the bits it gets as a prompt token, at 0.84–0.91× throughput. DeepSeek-V4 [6] gives up split-KV and most split-K, and spreads one sequence over several SMs with a second attention kernel engineered to match the single-SM order. These systems remove batch dependence by pinning the partition or by engineering one kernel to reproduce another's order. We leave the partition free at the program's associativity points, which needs no matching and no thread-block clusters, and we make prefill and decode agree.

Deterministic attention. Fixed absolute key splits merged in order appear in mlx-batch-invariant [7] (256-key splits with a per-key online softmax on Apple GPUs; prefill 3.9–4.4× slower than stock, generation at 0.47–0.85×) and in a llama.cpp draft [8]. Our fold groups share the idea; what we add is the sequence *inside* the group — an online softmax over 32-key tiles that a FlashAttention-2 inner loop and a group-parallel decode kernel execute as they are — together with a projection program that leaves the GEMM tile free, inside a serving engine with a prefix cache. Vosti [9] specifies and verifies deterministic inference with one ordered online-softmax loop per query: our program with a single group of unbounded length, which forbids splitting long decode contexts. FoldAttention [10] replaces the running maximum with a declared reference for fast decode and deterministic backward. FlashDecoding [11] and FlashInfer merge split partials, but their split boundaries follow the schedule.

Bitwise GEMM. Closest in spirit is Yang et al. [12], who describe a GEMM's bitwise behaviour by a descriptor of its reduction order (instruction k, k-loop order, split-K cuts, partial and merge precision), observe that output tiling only moves work between threads and leaves each accumulator's chain intact, reconstruct cuBLAS's descriptor from black-box measurements, enforce chosen orders in Triton, and verify bitwise equivalence of compiled kernels statically on PTX and AMDGCN. Our GEMM program is such a descriptor, designed for serving rather than recovered from a library: its segment count depends on the weight alone, so it is invariant to M and to tensor-parallel sharding, and every variant in a family of hand-written, CUTLASS and stream-K kernels realises it. We extend the same discipline to attention, for which [12] gives only a theoretical description, and carry it end to end through an engine, a trainer and tensor parallelism. Cooper et al. [13] make GEMM order a function of the problem shape and accumulate in explicit IEEE fp32, which yields identical bits across Ampere, Ada and Hopper. TBIK [14] makes tensor-parallel all-reduce orders invariant; CoRun [15] pads to fixed shapes; LLM-42 [16] verifies and rolls back nondeterministic steps; reproducible summation [17] makes sums order-independent at higher cost. A recent stream-K for convolution gradients [18] folds published partial tiles in a fixed order; our stream-K hands on the accumulator itself, so a cut chain equals the uncut one and no fold exists.

3 Canonical programs

Guarantee. For a fixed GPU architecture, engine build and model weights, every response is a bitwise function of the request's token ids, sampling parameters and seed. Batch composition, concurrency, cache state, chunking, decode-versus-prefill path, CUDA graphs, page size and restarts do not enter; with a declared $t_{\max}$, neither does the tensor-parallel degree of the projection program (§8).

Model of computation. We rely on four properties of the hardware and compiler:

- **(A1)** An MMA instruction computes each output element as a deterministic function of its row of A, column of B and accumulator input.
- **(A2)** `ex2.approx` (and `lg2.approx`) is a deterministic function of its input.
- **(A3)** The compiled code does not reassociate arithmetic we spell out with round-to-nearest intrinsics.
- **(A4)** An MMA whose accumulator input is not -0 does not return -0, as IEEE round-to-nearest addition would not.

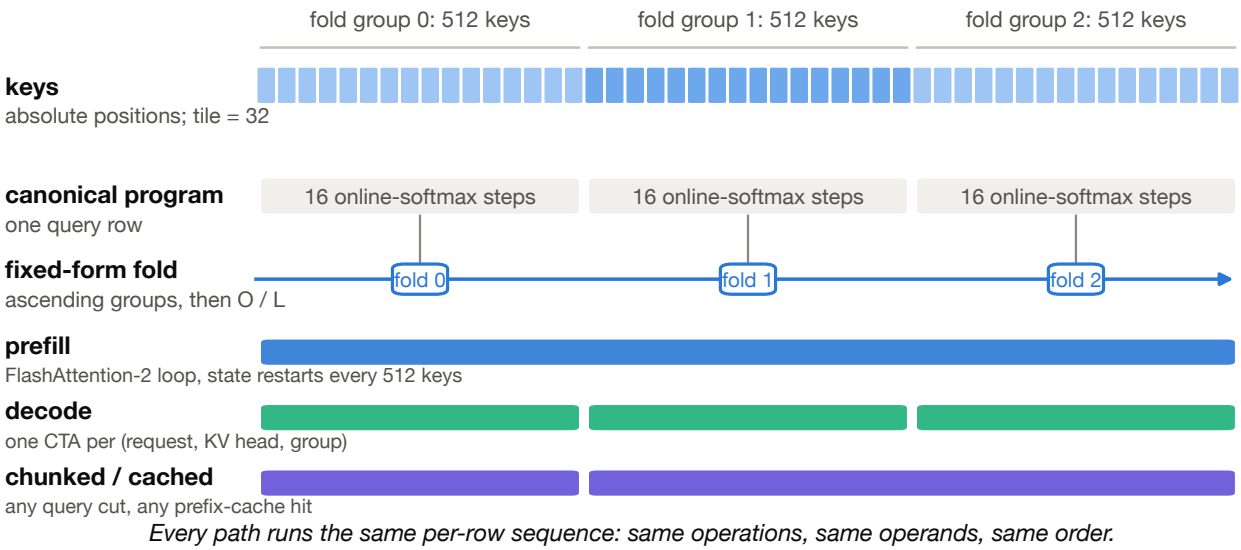


Figure 1. The canonical attention program. Keys are cut at absolute positions into 32-key tiles and 512-key fold groups. Each group runs 16 online-softmax steps from the empty state, and group partials are folded in ascending order. A prefill CTA, per-group decode CTAs, and any chunked or cached split all execute this same per-row sequence.

A1, A2 and A4 are properties of a GPU architecture, so the guarantee is stated per architecture. In practice the bits also agree across sm_80 and sm_89 (§6.2).

3.1 Attention

Keys are cut at absolute positions into tiles of $T = 32$ keys and fold groups of 16 tiles. For query row i at position p:

Scores. For each tile, $s_j = (q_i \cdot k_j) \cdot \text{scale} \cdot \log_2 e$, by MMA over the head dimension in ascending 16-wide steps from zero, masked to $-\infty$ for $j > p$.

Group. From the empty state $(m, d, o) = (-\infty, 0, 0)$, for each of the group's tiles in ascending order:

$$m' = \max\left(m, \max_j s_j\right), \qquad \mu = m' \text{ if } m' > -\infty \text{ else } 0, \qquad \sigma = 2^{m-\mu},$$

$$P_j = \text{bf16}\left(2^{s_j-\mu}\right), \qquad d \leftarrow \text{fma}\left(d, \sigma, \textstyle\sum_j P_j\right), \qquad o \leftarrow \text{fma}(o, \sigma, +0) + PV,$$

with the row sum in one fixed order and PV by MMA over the tile's two 16-key steps in ascending order.

Across groups. The partials (m_g, d_g, o_g) fold in ascending g into (M, L, O) from $(-\infty, 0, 0)$:

$$M' = \max(M, m_g), \quad \alpha = 2^{M-M'}, \quad \beta = 2^{m_g-M'}, \quad L \leftarrow \text{fma}(L, \alpha, d_g\beta), \quad O \leftarrow \text{fma}(O, \alpha, o_g\beta),$$

and the output is O/L. Two details make the program well defined on every path. *Masked steps are no-ops:* a prefill row may meet a tile entirely beyond its position, which decode never processes; there $\sigma = 1$, all P are 0, and since the rescale $\text{fma}(o, \sigma, +0)$ maps -0 to $+0$ and A4 keeps MMA results off -0, the state stays bit-identical, as if the step were skipped. *The first fold is exact:* folding into the identity returns the partial unchanged ($\alpha = 0$, $\beta = 1$), so a kernel may keep the first group in registers or read it from a workspace.

Lemma 1 (attention). If a kernel executes this program for every row it owns, in the stated order within and across groups, the output of row i is identical whatever rows share the launch, however groups are assigned to CTAs (as long as a CTA runs a whole group in order), wherever chunked prefill cuts the query range, whether keys come from a prefix cache or were just written, and whether the row is a decode row or a prefill row.

Proof sketch. Every floating-point operation applied to row i, and every operand it reads, is fixed by p and the keys and values at positions $\le p$. A kernel's freedom lies in which rows, groups and tile slices it co-schedules. Splitting a tile across warps partitions S by key columns and O by output columns; it changes neither an element's MMA chain nor the exact maximum, and the row sum is recomputed in the canonical order. □

The tile is what prefill streams through shared memory per online step. The group size trades decode parallelism against prefill work; on these kernels' predecessor, of 512, 1024 and 2048 keys, 512 was fastest for decode at batch ≤ 8,

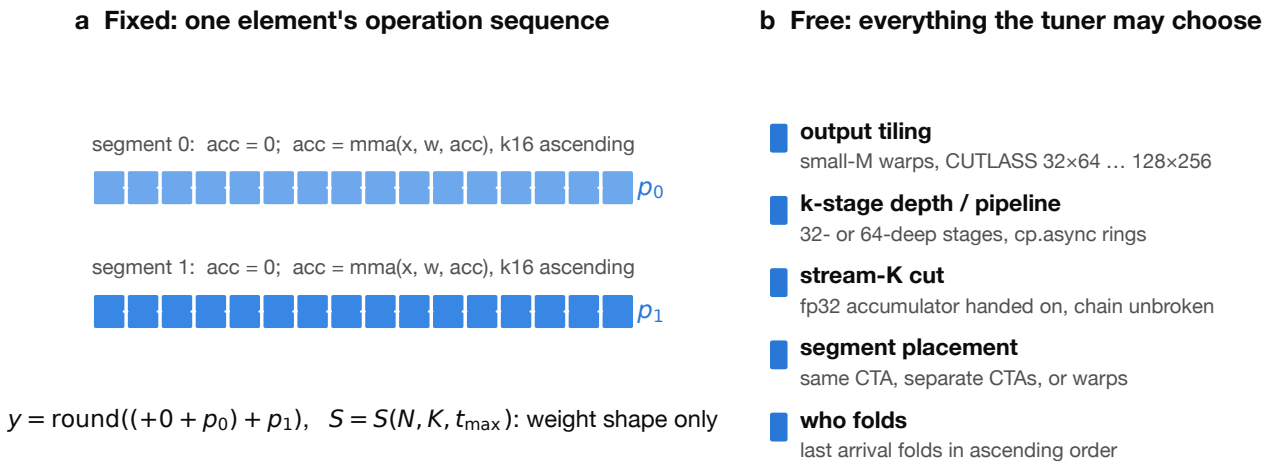


Figure 2. The canonical GEMM program. (a) Each output element runs one MMA chain per K segment over 16-wide k steps in ascending order from zero, and the segments are folded from +0 in ascending order; S depends on the weight shape and the declared $t_{\max}$ only. (b) Everything else is a free choice of the kernel and the autotuner.

for shared prefixes and for short extends, at no measurable cost to long prefill, and gives decode 1–64 CTAs per KV head at batch 1 for 0.5K–32K contexts.

3.2 GEMM

K is cut into $S = S(N, K, t_{\max})$ equal segments. Segment s accumulates its 16-wide k steps in ascending order by MMA from zero into p_s, and

$$y_{mn} = \text{round}\Big(\big(\cdots\big((+0 + p_0) + p_1\big) + \cdots\big) + p_{S-1}\Big).$$

With $t_{\max} = 1$ the rule gives $S = 2$ for $N \le 1024$, $K \ge 2048$ and K divisible by 512 — Qwen3-0.6B's o and down projections — and $S = 1$ otherwise.

Lemma 2 (GEMM). Any tiling of M and N, any assignment of (output tile, segment) pairs to CTAs or warps, any choice of which CTA folds a tile's segments, and any cut of a segment's chain between workers that hands the fp32 accumulator from one to the next yield identical rows.

Segments are deterministic split-K. Split-K is nondeterministic in existing engines because the split count follows M and the SM count and the partials are combined by atomics or in arrival order; fixing S by the weight and the fold by segment index removes both. The rule spends segments only where split-K pays: narrow projections with long K, which at decode have too few output tiles to occupy every SM.

3.3 The numerical contract

The two programs make the bits a function of a short list of parameters. Everything on the list is fixed per deployment, as much a part of the model's numerical definition as its dtype; everything off the list is a free performance choice.

Table 2. The numerical contract. Changing a left-column entry defines a different model numerically, like changing the dtype; right-column entries never change a bit.

fixed: defines the bits	free: chosen per launch
MMA instruction: `mma.sync.m16n8k16`, bf16 inputs, fp32 accumulate	output tile shape, warps per CTA, CTA count
k order: ascending 16-wide steps within a segment, from zero	k-stage depth, pipeline stages, `cp.async` ring sizes
segment count $S(N, K, t_{\max})$ and the fold $(((+0 + p_0) + p_1) + \dots)$	which CTA or warp runs a segment, and which folds
attention tile (32 keys), fold group (512 keys), fold form	stream-K cut points (accumulator handed on in fp32)
explicitly rounded softmax, fold and log-softmax arithmetic	rows per CTA, group-to-CTA mapping, split count
declared tensor-parallel maximum $t_{\max}$	actual tensor-parallel degree t

$t_{\max}$ is a contract parameter, not a tuning knob. S depends on $t_{\max}$ because a column shard of a wide weight at

large t is as narrow as a small model's projection and needs segments for the same reason (§8). Two deployments that declare different $t_{\max}$ therefore compute different bits, exactly as two deployments in different dtypes do; a deployment that fixes $t_{\max}$ gets identical bits at every degree t, including $t > t_{\max}$. The engine fixes $t_{\max} = 1$ — it exposes no flag to change it — which keeps every Qwen3 and Qwen3.5 projection at the segment count it always had. A tensor-parallel launch never derives S from its local shard: every launch receives the whole weight's S as an argument, so a shard runs, for each of its output elements, exactly the chain the unsharded weight runs (§8).

Every kernel realises the same instruction-level program. Following the descriptor view of [12], a GEMM's bits are determined by the instruction that accumulates, the order and grouping of its k steps, the handling of a K remainder, and the precision and order of any merge. Our kernels hold each of these fixed by construction:

- *One instruction.* Hand-written kernels issue the single inline-PTX `mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32`; the CUTLASS tiles are instantiated for sm_80 tensor ops with instruction shape `GemmShape<16, 8, 16>` and CUTLASS's default multiply-add, which lowers to the same instruction. Attention's $QK^\top$ and PV use it too.
- *No K remainder.* Every kernel requires its segment length to be a whole multiple of its k tile and refuses the launch otherwise; there is no tail iteration to place first or last, and a refused kernel is simply not a tuning candidate.
- *No hidden rounding.* A segment's chain stays in fp32 registers between MMAs. Stream-K stores and reloads the fp32 accumulator bit-exactly, and segment folds are explicit fp32 additions of stored fp32 partials in ascending order — pure additions, with nothing a compiler could contract into an FMA. The build uses no fast-math flags.
- *Checked, not assumed.* `test_gemm.py` compares every kernel id against two reference kernels on every output row of every projection shape at 14 values of M from 1 to 4096, including tile-edge values 7, 9, 17, 33 and 65; mutation controls (§6.7) show the check rejects a wrong segment count and cuBLAS's own M-dependence. The verification is dynamic and exhaustive over the shapes the engine runs; a static checker in the style of [12] would extend it to unseen shapes.

3.4 Composition

The remaining operators are row-wise or elementwise. RMSNorm (with the residual add) reduces within one row in a fixed order, and q/k-norm runs one warp per head; RoPE, SiLU-and-mul, the embedding gather and the KV store are elementwise. The sampler's softmax reduces each vocabulary row in one block whose access pattern depends only on the row's 16-byte alignment, the same for every row because $4|V|$ is a multiple of 16 ($|V| = 151{,}936$). Greedy argmax over bf16 logits combines blocks in a batch-dependent way but selects the first maximal index, a total order, so its result is batch-independent.

The sampled token's log-probability is a new row reduction, so we fix it explicitly: one CTA of 1024 threads per row; thread j reads columns $j, j+1024, \ldots$ in ascending order for the exact maximum and then for $\sum 2^{(z-m)\log_2 e}$; per-thread sums are added in a fixed shared-memory tree; and the result is $(z_t - m) - \log_2(\Sigma)\ln 2$. Only explicitly rounded operations and `ex2`/`lg2` are used, so a trainer that links the same header computes the same value.

Theorem. In deterministic mode, every activation, KV entry, logit and log-probability computed for token t is a function of tokens $0..t$ only. With per-row Philox streams keyed by (seed, position), sampled tokens are a function of tokens $0..t$ and the seed only.

4 PI-Attn kernels

All kernels target sm_80/sm_89. One header holds the canonical pieces — the online step, the fold functions built from `__fmaf_rn`, `__fmul_rn` and `ex2.approx`, and the finish. Kernels choose how rows, groups and tile slices map to warps and CTAs; they never change an element's sequence.

4.1 Prefill and extend

The design follows FlashInfer's scaffold. A CTA has 4 warps of 16 query rows; its rows are the (token, head) pairs of one KV head's GQA group, so each K/V tile is loaded once for the whole group. Q, K and V live in 128-byte-swizzled shared-memory tiles — a 64-row Q tile plus two 32-key K slots and two V slots, 48 KB — so two CTAs share an SM. A software pipeline computes $S_{t+1} = QK_{t+1}^\top$ while tile t runs its online step and PV_t; S_{t+1} depends on nothing of tile t, and every row still runs its online steps in tile order. K and V load a full iteration ahead behind one barrier per tile. Positions past the KV length are zero-filled without a global read. Each warp folds finished groups into a second register state; with 16 rows per warp both states fit in registers.

On sm_80 a second variant keeps Q in registers and moves the fp32 group totals, touched once per 512 keys, to shared memory (80 KB, still two CTAs on the A800's 164 KB), cutting the per-tile shared-memory traffic of the Q operand. The same variant also runs with 8 warps (128 rows) per CTA, which copies each K/V tile once for twice as many rows; it is 5–8% faster on long and batched prefills and up to 12% slower on short extends, so the launcher takes it only when 128-row CTAs fill two waves. Sixteen-row warps are deliberate: 32-row warps need the fp32 group totals outside registers, which halves occupancy to 4 warps per SM, and the built variant ran 1.4–1.8× the shipped kernel's time.

For long prefixes with few query rows, a wave-model planner splits a row tile's groups across CTAs, each writing group partials that a merge kernel folds. On the RTX 4090 D the planner cut a 2048-token chunk's attention by 9%; on the A800 an unsplit kernel was always faster, so sm_80 keeps it.

4.2 Decode

One CTA of 4 warps runs one (request, KV head, group). Per tile, each warp computes an 8-key slice of S; the tile maximum is exchanged through shared memory; each warp rounds its slice of P; warp 0 recomputes $\sum P$ in the canonical order; each warp accumulates four of the sixteen output column tiles. Decode is a single kernel: a request with one group finishes in place, since the first fold is exact; otherwise each warp writes its slice of the group partial and counts its arrival on a per-(request, KV head, warp) counter, and the last warp to arrive folds that slice over all groups in ascending order. Which CTA folds is a scheduling accident; the folded values and their order are not. No CTA-wide barrier follows the arrival — an earlier version that parked the CTA on thread 0's atomic doubled kernel time under L2 pressure. The workspace is batch × groups rows and the grid depends only on the batch and the page-table width, so CUDA-graph replay needs no host planning. The group is the slowest grid index, so every working CTA is dispatched before the empty ones and the KV heads of a token are read together; on the A800 this cut Qwen3-1.7B's decode attention at 8 requests from 1001 to 842 µs per step.

4.3 The row-invariant GEMM family

Every variant realises the segmented program of §3.2, so all are bitwise identical (Lemma 2), and an autotuner picks the fastest per (N, K, M-bucket) with no numerical effect:

- **Small-M kernels** with deep `cp.async` pipelines (128–256 k per stage) that fold segments in registers.
- **Balanced kernels** for $M \leq 64$ that split (column tile, segment) work items contiguously over one CTA per SM. Decode GEMMs stream weights, and fixed 32-column tiles leave some SMs with twice the bytes of others.
- **CUTLASS 2.x tiles** from 32×64 to 128×256 with 32- or 64-deep k stages. On segmented shapes a derived kernel calls the mainloop once per segment from a zero accumulator and folds into a register total before the stock epilogue. At $M = 128$–512 the 64-deep stages halve the mainloop's barriers per k and took several projections from 1.1–1.3× cuBLAS's time to 0.85–1.05×.
- **Deterministic stream-K.** Some projections have too few output tiles to divide evenly over the SMs. A stream-K kernel splits the (tile, k-step) pairs of all tiles evenly over one worker warp per (SM, warp slot); a tile cut between workers w and $w+1$ stays one chain — w runs the first k steps from zero, stores the fp32 accumulator and raises a flag, and $w+1$ continues the same MMA chain from it. The result equals the uncut chain bit for bit. Each worker runs its handed-off prefix first and its continuation last, and only ever waits on the next-lower worker's prefix, which never waits; with at most one CTA per SM, all are resident and deadlock is impossible. This is split-K without a reduction. A variant runs chains over two adjacent column tiles that share each X fragment, bringing 1.7B's down projection at $M = 8$ to 22.7 µs against cuBLAS's 21.2. The same scheme runs at CTA granularity over CUTLASS tiles for mid M, and refuses to launch unless all CTAs can be resident.
- **Segment-parallel and segment-split kernels** for segmented shapes. The segment-parallel kernel gives each (tile, segment) pair its own CTA, and the last CTA of a tile to arrive adds the fragments to +0 in ascending segment order; no CTA waits on another. At $M \leq 8$ the segment-split kernel gives each column tile one CTA and each segment one warp with its own `cp.async` ring, folding through shared memory; on the RTX 4090 D it runs 0.6B's o projection in 6.01 µs at $M = 1$ against cuBLAS's 6.57.

Tuning that sees what the engine sees. Timing a microsecond kernel after an L2 flush misranks candidates, and kernels shorter than about 10 µs are launch-bound when launched from Python. The tuner therefore times bursts of back-to-back launches over a ring of weight copies larger than L2, captured in a CUDA graph, so every launch streams its weights from DRAM as consecutive layers do.

5 Engine integration

We implement the mode in llm_infer_rust, a Rust/libtorch serving engine with a radix prefix cache, chunked prefill, mixed prefill/decode batches, decode CUDA graphs, segmented prefill graphs and CPU/GPU overlap. `--deterministic` disables none of them; it changes three things.

1. **Attention** uses PI-Attn for prefill, extend and decode.
2. **Projections**, including the LM head, use the row-invariant GEMM.
3. **Sampling** uses a batched fork of FlashInfer's top-k/top-p kernel with each row's Philox stream keyed by (request seed, position) rather than batch row. Requests without a seed get a random one, echoed in the response, so any output can be replayed.

A completion request with `logprobs: 0` also receives each sampled token's log-probability from the fixed-order kernel of §3.4. It runs only in steps where a row asks for it, reads the logits the sampler already holds, and returns in the same asynchronous copy as the token ids, so the overlap pipeline is unchanged.

After graph capture, the engine tunes every projection shape for every M bucket up to the largest forward — the buckets are the CUDA-graph capture sizes, so every graph runs the kernel timed at its own M — on random operands, since on zeros some tiles run up to 7.5% faster and would be misranked. Tuning takes 4.0 s (0.6B), 7.8 s (1.7B), 10.1 s (4B) and 15.5 s (8B) at startup on the A800. Because tuning follows capture, a tuned kernel may need a larger CUTLASS scratch buffer than the captured graphs recorded; a grown buffer keeps its predecessor alive for the life of the process, since the graphs still launch into it.

For ablation, `--attention-backend pi` runs PI-Attn with cuBLAS projections, and `--row-invariant-gemm` runs the row-invariant GEMM and deterministic sampler with FlashInfer attention.

6 Evaluation

Setup. One A800-SXM4-80GB (sm_80, 108 SMs, 40 MB L2; driver 535.129.03, CUDA 12.6, PyTorch 2.13, FlashInfer 0.6.18) running Qwen3 0.6B, 1.7B, 4B and 8B in bf16, at stock clocks (1410 MHz in 78% of loaded samples, never below 1170 MHz). *Original* is llm_infer_rust with FlashInfer attention, cuBLAS projections and its default sampler; *PI* is the same binary with `--deterministic`. Both run radix caching, chunked prefill (2048 tokens), mixed batches, CUDA graphs and overlap. Same-GPU baselines are SGLang 0.5.5.post3 (`--enable-deterministic-inference`), vLLM 0.11.2 and vLLM 0.17.1 (`VLLM_BATCH_INVARIANT=1`). An RTX 4090 D (sm_89) supplies the cross-GPU comparison and §6.6. All numbers come from one scripted run (`eval/run_all.sh`), with every bitwise check repeated on the final engine; the artifact appendix maps each result to its files.

6.1 Determinism

detcheck sends token-id prompts of 24–2600 tokens (256 output tokens, `ignore_eos`) and compares, request by request, the token ids *and* the fp32 bits of every token's log-probability. Figure 3 shows the result on Qwen3-1.7B within one server instance.

PI-Attn passes every check. So does vLLM 0.17.1's batch-invariant mode, except the seeded variant of decode versus re-prefill: its sampler keys the random stream by the request's start rather than by (seed, position), so a re-prefilled request draws different numbers, while the greedy variant, which compares the same log-probabilities without a sampler, passes. SGLang's deterministic mode removes most batch dependence of *tokens*, but its log-probabilities still change with concurrency (5 of 16 requests) and across 30 of 32 requests that share a prefix, and no engine other than PI and vLLM 0.17.1 batch-invariant makes decode agree with prefill. Log-probabilities are the stricter test: at token level the original engine looks consistent on decode versus re-prefill (0 of 24) and on prefix-cache hits (0 of 8), and SGLang's deterministic mode differs on only 1 of 24; with log-probabilities, all 24 differ for both.

detcheck passes on all four models with log-probabilities compared: 200 repeats of one prompt give one distinct completion, and seeded sampling at concurrency 4 and 16 matches the sequential reference in all 16 requests. **Across restarts and configurations** (Qwen3-0.6B, 16 greedy and 16 seeded-sampled requests against references from a base server), PI is identical under chunk size 512, chunked prefill off, CUDA graphs off, at most 4 running requests, overlap off, page size 32, and a plain restart. The original engine fails 11–13 of the 32 comparisons in each, including the plain restart.

Evaluation becomes a function of the model. GSM8K 5-shot through lm-eval at client concurrency 1 and 16:

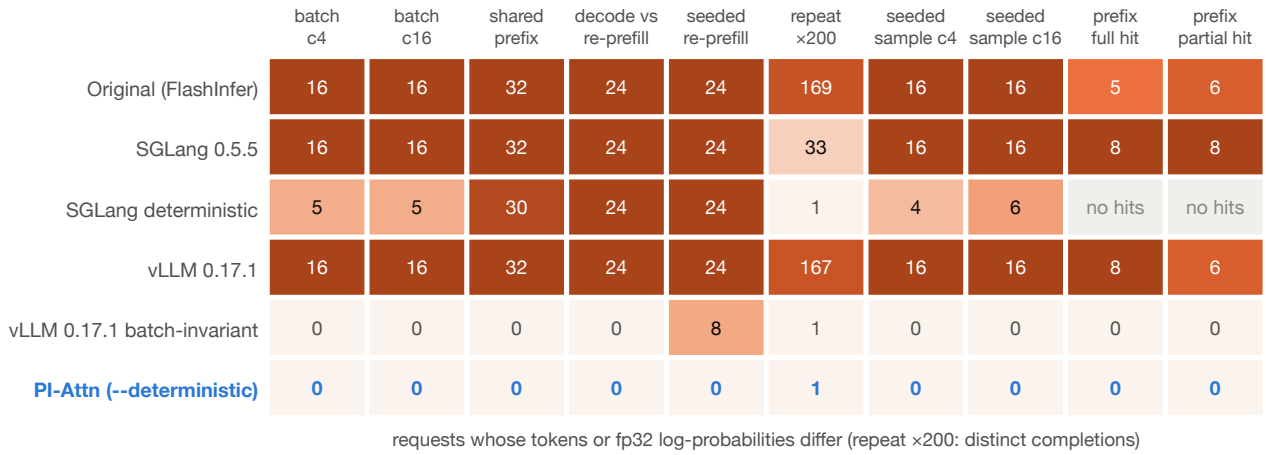


Figure 3. Requests whose tokens or log-probabilities differ from the reference (Qwen3-1.7B, A800). Batch: the same 16 requests at concurrency 4 and 16. Shared prefix: 32 requests sharing a prompt prefix in one batch. Decode vs re-prefill: a request's continuation after re-submitting its first k generated tokens as prompt. Repeat: distinct completions over 200 repeats of one prompt. Prefix hit: full and partial prefix-cache hits. SGLang's deterministic mode disables the radix cache on this path, so it has no hits to compare.

Table 3. GSM8K at two client concurrencies.

model	mode	strict acc. c=1	c=16	generations differing, c=1 vs c=16
Qwen3-1.7B (1319)	original	69.37	68.69	888
Qwen3-1.7B (1319)	PI	69.07	69.07	**0**
Qwen3-8B (500)	original	89.80	89.80	322
Qwen3-8B (500)	PI	90.20	90.20	**0**

With the original engine two thirds of the 1.7B generations change with client concurrency, and its score moves by 0.68 points — more than the gap between the two modes. The original engine's score is a sample; PI's is a value.

6.2 Same bits on two GPU architectures, and accuracy

A kernel-hash harness runs 80 GEMM cases (20 projections × M = 1, 8, 128, 1024) and five attention cases on fixed inputs and hashes the outputs on both GPUs. Every PI kernel case, and every PI GSM8K generation, is bitwise identical between the A800 (sm_80) and the RTX 4090 D (sm_89) (Figure 4); cuBLAS agrees on 14 of 80 cases, FlashInfer on 1 of 5, and the original engine's generations differ in 906–910 of 1319 (1.7B) and 325–343 of 500 (8B). The 4090 D hashes come from the earlier engine; the kernels added since change the schedule, not the program, and on the A800 every new kernel is checked bit for bit against the two reference kernels that produced the cross-GPU GEMM hashes.

Accuracy is unchanged. Top-1 agreement with an fp32 HF reference over 1,280 teacher-forced positions per model:

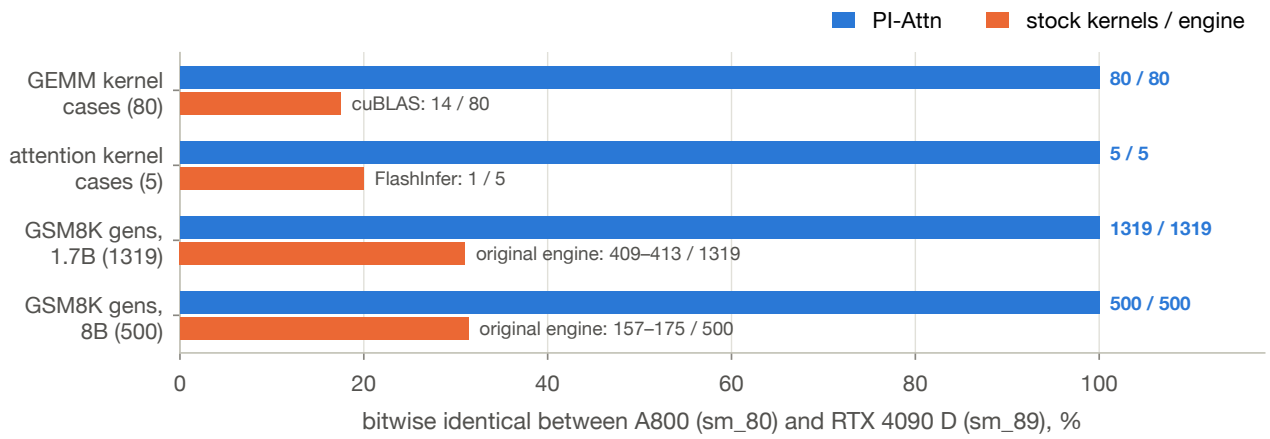


Figure 4. Bitwise identity between an A800 and an RTX 4090 D: kernel cases and end-to-end GSM8K generations (both client concurrencies; the original engine's range is over the two).

Table 4. Top-1 agreement with fp32.

	0.6B	1.7B	4B	8B
original	99.53%	99.14%	99.30%	99.53%
PI	99.61%	99.30%	98.91%	99.53%

The modes differ by at most 5 positions per model, in either direction, all at positions whose fp32 top-2 margin is below 0.17 logits; PI's generations are identical on both GPUs. Accuracy is asserted in the tests: against float64, PI-Attn's mean error is below FlashInfer's, and its maximum error no larger, on every GQA geometry and query scale tested.

6.3 Rollout log-probabilities equal the trainer's

An RL trainer recomputes the log-probability of every sampled token with its own forward and divides by the rollout engine's value to form an importance ratio. When the two programs differ, the ratio drifts from 1 on tokens the policy never changed, and methods such as truncated or masked importance sampling exist to absorb that noise. Row invariance removes it at the source, provided the trainer runs the same canonical program.

The trainer side is the engine's Qwen3 forward in PyTorch: the PI GEMM, PI-Attn causal prefill over a paged cache, the engine's own RMSNorm, q/k-norm, RoPE and SiLU kernels compiled from its source with its flags, and the same log-softmax. We send 64 GSM8K questions per model (chat template, 256 tokens, temperature 1 with a per-request seed, every fourth request greedy) to the deterministic engine at concurrency 16, then score exactly those sequences with two trainers: the PI forward, each sequence alone and in two random packings of 2–8 sequences per micro-batch; and HF transformers in bf16 with SDPA and an fp32 log-softmax.

With the PI forward, every one of the 55,593 completion tokens has the engine's log-probability bit for bit, whichever packing the trainer uses, so every importance ratio is exactly 1 (Figure 5). The engine produced those tokens by decode in batches of up to 16 requests under chunked prefill; the trainer recomputes them in one prefill per micro-batch. The HF trainer matches on 1.0–40.8% of tokens and misses the 1% ratio band on 10–22%, with ratios from 0.49 to 2.51.

6.4 Cost

End to end (Figure 6a), PI runs at −5.9% to +14.2% of the original engine across 40 workload points (four models, five workloads, greedy and sampled at $T = 0.8$, which stay within 1.7 points of each other). At batch 1 it is *faster* on 1.7B, 4B and 8B, by 5.7–13.8%, where PI's single-kernel decode attention and, on 1.7B, its stream-K down projection both beat the originals (see *Components*); 0.6B at batch 1 is 3.4% slower (§6.5). Batched workloads sit within ±2.5% except 1.7B at 8 requests (−5.8%) and three 0.6B points (+4.3% to +7.1%).

Components. Running each half of the mode alone shows where the cost and the gains come from:

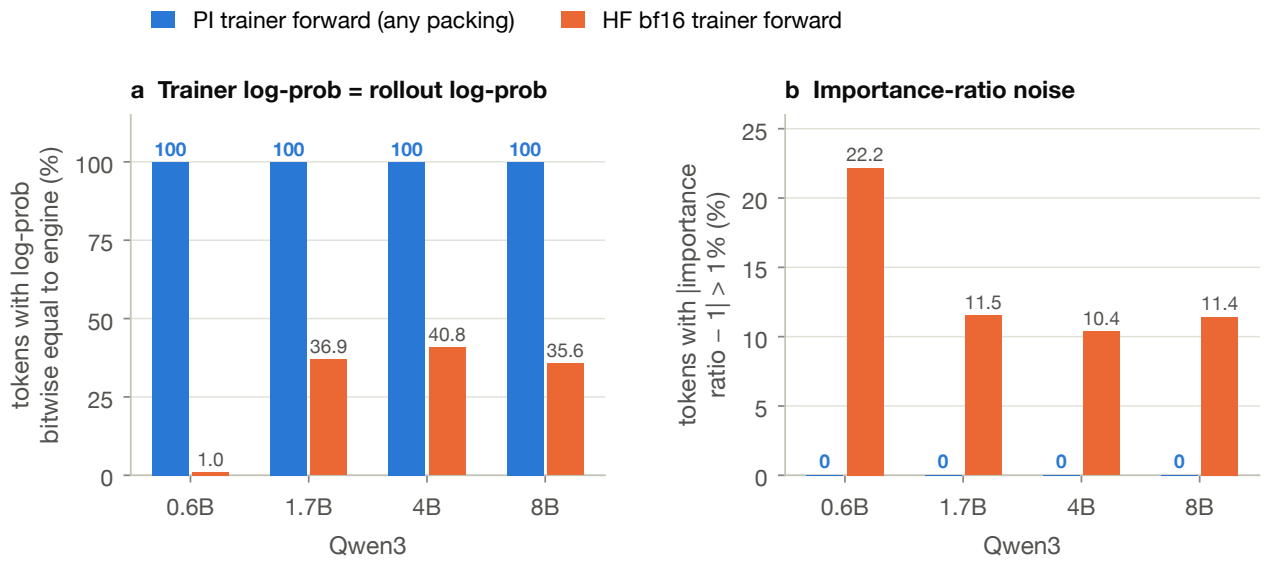


Figure 5. Trainer log-probability versus the engine's, per token. (a) Fraction of tokens whose log-probability equals the engine's bit for bit. (b) Fraction whose importance ratio deviates from 1 by more than 1%. HF: each sequence scored alone; batches of 8 give 10.6–22.1% in (b).

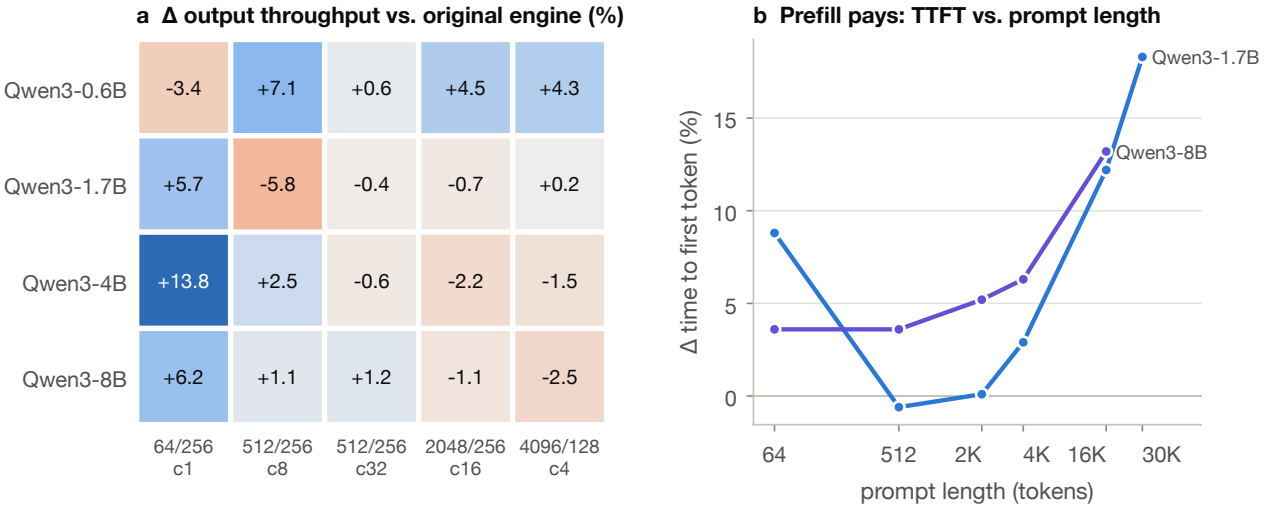


Figure 6. (a) End-to-end output throughput of PI relative to the original engine (greedy; median of three interleaved launches). (b) Change in time to first token for the long-context workloads of Appendix A, by prompt length (each point at its own concurrency; Qwen3-1.7B up to 30K tokens, Qwen3-8B up to 16K).

Table 5. Component ablation (output throughput vs. original; workload = input / output tokens / concurrency).

model	workload	original tok/s	PI attention only	row-invariant GEMM only	both
0.6B	64 / 256 / c1	513.5	+3.3%	−6.1%	−2.9%
0.6B	512 / 256 / c32	7454.7	+1.3%	−0.8%	+0.4%
0.6B	512 / 256 / c128	11641.0	−0.3%	−1.4%	−1.7%
1.7B	64 / 256 / c1	282.7	+1.8%	+3.9%	+5.9%
1.7B	512 / 256 / c32	5080.5	+1.1%	−1.6%	−0.6%
8B	64 / 256 / c1	89.9	+4.7%	+1.4%	+6.8%
8B	512 / 256 / c32	1813.7	+0.5%	+0.5%	+1.1%

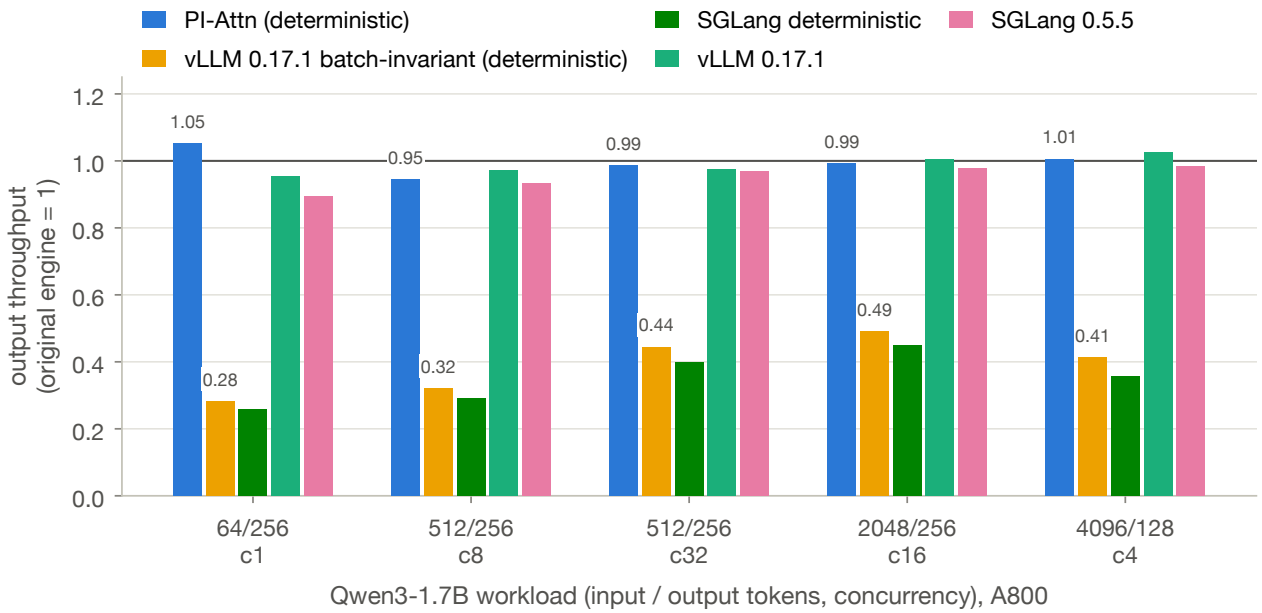


Figure 7. Output throughput on Qwen3-1.7B relative to the original (non-deterministic) engine. The two deterministic baselines give up 51–72% (vLLM batch-invariant) and 55–74% (SGLang deterministic) of the original engine's throughput; PI-Attn gains 5% at batch 1 and gives up at most 5.5% elsewhere. Absolute numbers are in Appendix A.

PI attention alone never costs more than 0.3% on these decode-heavy points and gains 1.8–4.7% at batch 1. The row-invariant GEMM costs up to 1.6% at 32 and 128 requests and 6.1% on 0.6B at batch 1, whose o and down projections are its slowest shapes, and gains 1.4–3.9% on 1.7B and 8B at batch 1. The two halves roughly add.

Prefill is where the canonical program pays on this GPU (Figure 6b). Time to first token rises by up to 6.3% on 2K–4K prompts and by 12–18% on 16K–30K prompts, and a single 30000-token request loses 9.4% of its throughput. Shared-prefix workloads still gain throughput (+1.2% to +8.1%) despite the higher TTFT. At high concurrency, decode-heavy points at 64–128 requests stay within −2.0% to +7.5%; prefill-heavy 0.6B is the expensive case, losing 8.0% of throughput on 2048-token prompts at 64 requests and adding 18.9% to time to first token at 128 requests, where each step is a large batched prefill. The same points cost 2.9% and 2.3% on 1.7B. Appendix A lists every long-context and high-concurrency point.

Against other engines (Figure 7), PI is within −2.8% to +10.4% of vLLM 0.17.1 without determinism — behind on three workloads by 1.2–2.8% — and ahead of SGLang on all five. The deterministic modes of the other engines pay far more: vLLM 0.17.1's batch-invariant mode, the only other engine that passes detcheck's numeric checks, runs at 30–49% of vLLM's own throughput, and PI delivers 2.0–3.7× its throughput — measured outside that mode's documented compute-capability range, so its cost on Hopper may differ; SGLang's deterministic mode, which also disables the radix cache, runs at 29–46% of SGLang's.

Kernels. Attention is timed kernel against kernel by replaying a CUDA graph of at least ten calls, each over another layer's KV cache from a ring larger than L2, so neither launch overhead nor a warm cache enters; FlashInfer is planned for each exact shape.

- *Decode,* PI's time relative to FlashInfer's: 0.91–1.06× the faster of FlashInfer's CUDA-core and tensor-core decode at batch 8–32; 0.73–0.85× with 8 or 32 requests sharing a 4096-token prefix; 1.05–1.10× at batch 128; 1.03–1.10× at batch 1 with 8K–32K keys, and 1.92× (24.8 vs 12.9 µs) at 1024 keys, where two groups give two CTAs per KV head while FlashInfer splits finer. With FlashInfer's split size fixed at 2048, its batch-invariant configuration, PI is 5.2× faster at batch 1 and 32K keys. In the engine at batch 1, PI's decode takes 12.9 µs per layer on 8B against 30.7 µs for FlashInfer's decode and merge.
- *Prefill and extend,* PI's time: 1.12–1.80× FlashInfer's on 17 of 18 shapes (0.55× on the 18th): long single prompts 1.22–1.34×, batched short prompts 1.42–1.52×, extends over 4K–16K prefixes 1.22–1.80×.
- *Projections,* best PI kernel against cuBLAS over the 20 Qwen3 projections with every launch streaming from DRAM: 0.78–1.37× at $M = 1$ (only 0.6B o and down above 1.05×); 0.43–1.26× at $M = 8$–64; 0.81–1.33× at $M =$

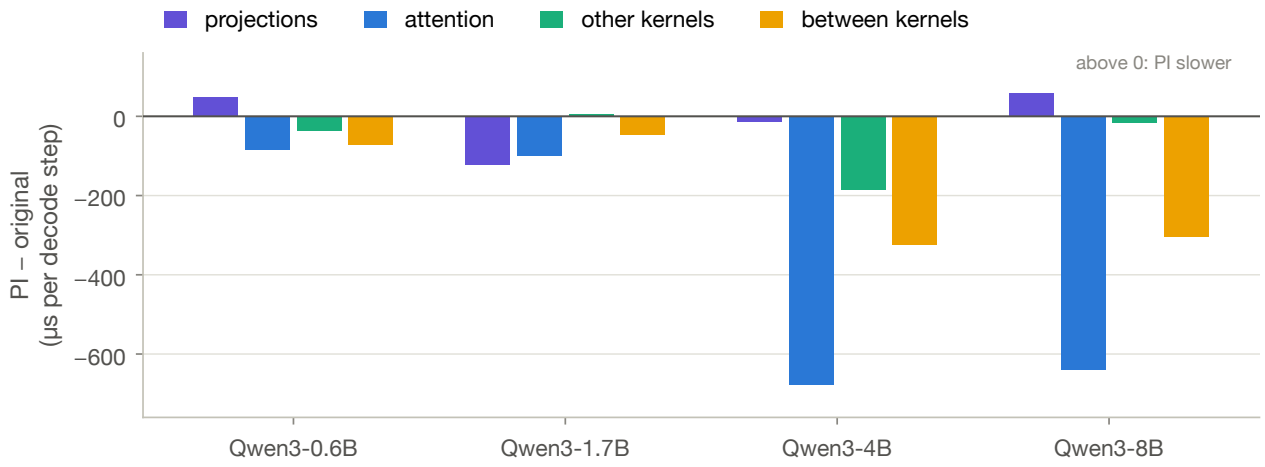


Figure 8. Per-step time of PI minus the original engine at batch 1, by kernel category, from a node-level trace of greedy decode.

128–512; 0.76–1.22× at M = 1024–2048. A whole decode step's projections replayed as one graph take 1.11×, 0.77× and 1.03× cuBLAS's time at M = 1, 8 and 32 on 0.6B.

6.5 Where the cost goes

Batch 1 (Figure 8). Attention is faster on every model, by 86–679 µs per step, because PI's decode is one kernel whose single-group requests finish in place. PI's projections are faster per step on 1.7B and 4B and cost 0.5% of a step on 8B; on 1.7B the stream-K down projection beats cuBLAS (25.94 ⟶ 20.71 µs per launch). 0.6B's loss is in its two segmented projections, o and down: 9.25 and 10.66 µs against cuBLAS's GEMV at 7.33 and 9.28. Their 4 and 6 MB of weights stream in 2–3 µs, so both kernels are latency-bound, and the gap is the round trip of the two-segment fold — the price of the contract's $S = 2$ for this shape, paid by every fold and hand-off scheme we built.

Prefill on full-rate tensor cores. The program is the same on every GPU; what changes is what it costs. The RTX 4090 D runs fp32-accumulate MMA at half rate, so MMA time dominates prefill and hides the two costs the program imposes: the per-32-key online step, a serial chain of shuffles and `exp2` between the $QK^\top$ and PV MMAs, and the fragment traffic of 16-row warps. On the A800 the MMA runs at full rate and both surface. Pipelining S_{t+1} behind tile t's online step and 8-warp CTAs brought 30000-token TTFT from +41.8% to +18.3%.

Batched decode (1.7B, 8 requests): PI's kernels keep the GPU busy 332 µs longer per traced step, the wall step is 213 µs longer. Attention accounts for 222 µs — each working CTA runs its group's 16 tiles in order where FlashInfer splits more finely — and projections for 155 µs, mostly o and down, which cuBLAS runs as split-K with a reduction kernel and as K-sliced CTAs. Both change an element's k order; deterministic stream-K recovers part of the gap.

What remains is the price of the contract itself: the 32-key online step, the 16-tile chain of a decode group, and the k order that rules out split-K. Lowering it means choosing a different canonical program — a different tile or group size — and with it different bits. On the RTX 4090 D, where the MMA hides the first two, the same design costs at most 1.8% end to end.

6.6 RTX 4090 D

The 4090 D ran the full evaluation with an earlier engine. PI passes every detcheck check, at token level, on all four models and all eight server configurations, where the original engine fails 13–14 of 32 comparisons in each and SGLang's deterministic mode diverges on 14 of 32 shared-prefix requests. End to end, PI is *faster* than the original engine on 18 of 20 greedy points (up to +8.5%), costs 0.6% and 1.4% on the other two (batch 1 on 8B and 1.7B), and keeps time to first token within +1.6% on every long-prompt workload, 16K–30K tokens included.

Table 6. End-to-end throughput of PI vs. original on the RTX 4090 D (earlier engine).

model	64/256 c1	512/256 c8	512/256 c32	2048/256 c16	4096/128 c4
0.6B	+2.0%	+8.5%	+4.3%	+5.6%	+5.8%
1.7B	−1.4%	+5.7%	+4.9%	+3.4%	+3.9%
4B	+0.1%	+5.9%	+6.1%	+3.2%	+3.0%
8B	−0.6%	+1.5%	+3.3%	+0.6%	+0.9%

6.7 The tests can fail

A bitwise check that never fails proves little unless it can see a violation, so we mutate the kernels and require the tests to catch every mutant. Each mutant is a copy of the shipped sources with one change to one path. A split-prefill merge that folds groups in descending order is caught by the split check; prefill that closes a fold group every 256 keys instead of 512 by the decode-versus-prefill, split and head-shard checks. A decode merge that rounds twice instead of using one fma — a one-ulp fp32 difference — reached the bf16 output too rarely for the first version of the decode-versus-prefill check, which probed a dozen positions per request; the check now decodes every position of every request (about 4,300 positions in two random batchings) and catches it with 60 failures. On the GEMM side, cuBLAS run through the M-invariance check fails on 26 of 27 projections, and the fold check rejects a weight computed with one segment where its canonical count is two. End to end, detcheck compares every log-probability's bits, which is what exposes the original engine on prefix-cache hits.

Accuracy is asserted, not printed. Attention error against float64 must stay within 1.05× (mean) and 1.25× (maximum) of FlashInfer's single-prefill kernel on every GQA geometry and query scale; PI's mean error is below FlashInfer's everywhere (for 4096 keys: 8.1×10^{-5} vs 8.3×10^{-5} at query scale 1, 7.5×10^{-4} vs 7.7×10^{-4} at scale 6). Every fp32 GEMM output lies within the standard summation bound $(K+S)\,2^{-23} \sum_k |x_k w_k|$ of the float64 product, and every bf16 output is exactly the rounding of the fp32 one, for every projection of the four models.

7 Exact Gated DeltaNet decode

Hybrid models (Qwen3.5, Qwen3-Next, Kimi Linear) interleave full attention with Gated DeltaNet (GDN) layers. GDN is trained with a *chunked* forward — fla's `chunk_gated_delta_rule`: 64-token chunks, an intra-chunk triangular solve and a state carried between chunks — and served with a *recurrent* kernel that updates an fp32 state per token. These are two programs, so the sampler's log-probabilities are not the trainer's, and with speculative decoding a third kernel makes them depend on whether speculation is on. The principle of §3 applies unchanged: fix one canonical program per row and make every serving path execute it. For GDN the natural canonical program is the trainer's own chunk program.

The premise holds. fla's chunk program is row-causal and resumable bitwise: truncating the sequence, resuming from the fp32 state at any 64-token boundary, replaying a partial chunk token by token and varlen packing all reproduce the full forward's rows. fla's recurrent kernel differs on every row (relative L2 error 4.7×10^{-3}). On Qwen3.5 0.8B and 4B with row-invariant dense ops, chunk-exact decode equals the teacher-forced forward on all 724 scored positions, while recurrent decode flips the argmax at 0.8–1.2% of them: once dense ops are row-invariant, GDN is the entire remaining gap.

The kernel. A decode step computes the current token's row of the chunk program from the slot's state at the last 64-token boundary and the chunk's cached rows; once a chunk's 64th row is accepted, a fold advances the boundary state with the chunk kernel's own arithmetic. The state is read as a bf16 copy, and the fp32 state is recovered exactly from that copy plus its dropped 16 bits and one rounding bit, a lossless codec checked on every fp32 bit-pattern class. Memory per slot and layer is 4.33 MiB against the recurrent state's 2 MiB. Per layer and step on the Qwen3.5-4B shape, the kernel takes 2.01× vLLM 0.17.1's GDN decode time at batch 1, 1.40× at 8, 0.99× at 32 and 0.79–0.85× at 64–256.

Speculation is free and invisible. A verification window of up to 16 tokens computes more rows of the same chunk program, so its rows are the trainer's whatever the window. Rejected rows are overwritten by the next window, and a chunk is folded only after its 64th row is accepted, so rollback needs no per-draft state copies. vLLM's speculative path keeps 1 + drafts fp32 states per request (10 MiB per layer for 4 drafts); our window of 4 runs at 0.73–0.84× of its time at batch ≥ 32.

Inside vLLM 0.17.1. Prefill runs fla's chunk program, decode and verification run the exact kernel with state in vLLM's own pool, and the dense projections run PI-Attn's GEMM through vLLM's batch-invariant hook. A self-check

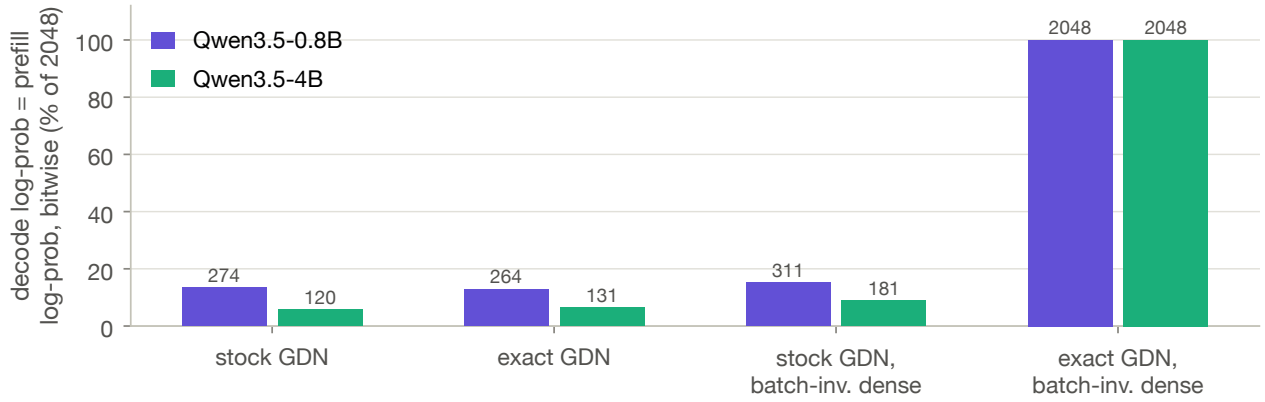


Figure 9. Decode log-probability equals prefill log-probability, bitwise, in vLLM 0.17.1 (2048 generated tokens per model). Only exact GDN together with batch-invariant dense projections makes decode the trainer's program.

generates 128 greedy tokens for 16 prompts, scores every sequence in one prefill, and counts generated tokens whose decode-time fp32 log-probability equals the prefill-time one (Figure 9).

With both on, decode equals prefill on 2048 of 2048 tokens with chunked prefill, prompts ending on a chunk boundary, slot reuse and preemption, and n-gram or MTP drafting leaves the output hash unchanged in each configuration; under seeded sampling decode still equals prefill on all 2048 tokens, though the sampled tokens, and so the hash, differ with speculation. With MTP drafting and chunked prefill together, the decode hash is still the unspeculated run's, but vLLM's own prompt log-probabilities change under that combination (decode equals them on 848 of 2048 tokens on 0.8B, and on 0 and 23 of 2048 on 0.8B and 4B with PI-Attn's GEMM), so the comparison there measures vLLM's prefill, not our decode. Throughput on Qwen3.5-4B: the bitwise configuration runs at 1.01× stock vLLM on 200/256-token requests, 1.08× with MTP drafting, 1.03× at 8 requests, 0.98× at one and 0.96× on prefill-bound work. With vLLM's own batch-invariant matmul in place of PI-Attn's GEMM, the same configuration drops to 0.27× at one request and 0.83× at 256: in vLLM, the cost of batch invariance lives in the dense projections, not in GDN. Exactness costs memory: the pool holds 2.0× fewer 472-token requests than stock without speculation, 0.96× as many 32K-token ones, and more than stock with speculation.

The trainer program is fla 0.5.0's chunk forward with four of its autotune configurations pinned (three of `l2norm_fwd_kernel` and one of `chunk_fwd_kernel_o` compute different bits from their siblings) and the same Triton version, on sm_80 and sm_89. vLLM vendors an older fla whose chunk forward differs, so the integration points vLLM at the trainer's fla.

8 Tensor-parallel degree as a canonical parameter

Tensor parallelism adds one more partition. Megatron splits the K sum of the row-parallel projections (attention output, MLP down) over the ranks and adds the partials with an all-reduce, so the summation order depends on the degree t. We keep every projection column-parallel instead: a rank holds the weight rows of its output columns over the whole K, the layer input is all-gathered first and the output after. An all-gather copies bits, so every output element runs the program it runs at $t = 1$ — for any t, including degrees that do not divide the head or column counts.

The cost is the chain floor. A narrow column shard keeps the whole K on few columns, and one MMA chain advances a 16-wide k step about every 28 ns on the A800, so a 1024-column shard of Llama-3-70B's down projection at $t = 8$ is bounded by its chains rather than by weight bandwidth. Segments shorten the chain, so the segment count must account for the narrowest shard the deployment will run: $S = S(N, K, t_{\max})$ doubles S while the shard $N/t_{\max}$ is at most $1024/S$ columns wide, up to 8, keeping segments at least 1024 long and multiples of 256. This is why $t_{\max}$ is part of the contract (§3.3). Every launch on any shard passes the whole weight's S, and the bits are identical for every t.

With ranks emulated on one GPU, the column shards of seven row-parallel shapes (Qwen3 4B and 8B, Qwen3.5-4B, 70B o and down) concatenate to the whole-weight result for $t = 1$–8 with every kernel id that runs the shard, and Qwen3-8B with real weights reproduces the $t = 1$ logits bitwise at $t = 2$, 3, 4 and 8 through prefill and 8 decode steps. The negative control, Megatron row-parallel o/down at $t = 2$, changes 77–87% of the logits, and still 79–93% with fp32 partials summed in fp32, where only the order differs. Per-rank GEMM time of the column shard against the Megatron

row shard of the same bytes is 1.02–1.26× at $t = 2$ for 70B and 1.12–2.4× at $t = 8$, worst on small models' narrow shards at $M \leq 32$. The column layout moves more bytes — four all-gathers per layer instead of two all-reduces, 1.63× the bytes per token for Llama-3-70B at $t = 8$ — a cost to be measured on a multi-GPU system.

9 Scope

The guarantee is per GPU architecture and build (A1–A4). The bits agree across sm_80 and sm_89 for every kernel case and every GSM8K generation we compared; Hopper's `wgmma` is a different instruction, and guaranteeing identity across architectures would mean explicit IEEE accumulation as in [13]. Cross-architecture identity also fixes the canonical program for every GPU: the segment rule and the fold groups are the same everywhere, which is what makes the A800's prefill cost the price of portability. The engine runs on one GPU with head dimension 128 and GQA groups up to 16; FP8 and MoE kernels, multi-GPU tensor-parallel execution and an RL training loop on top of the trainer forward are natural next steps. The workspaces and fold counters are per-device state that every launch leaves zeroed, which the engine's single forward stream guarantees.

10 Conclusion

Bitwise-reproducible serving does not require turning optimizations off. It requires choosing the floating-point program first and letting every kernel, every partition and every engine feature execute it. With attention cut into absolute 32-key tiles and 512-key fold groups and projections cut into shape-determined segments, prefill, decode, chunking, caching, batching, packing and even the tensor-parallel degree become schedule choices that cannot change a bit — and a trainer that runs the same program sees exactly what the rollout engine saw. On an A800 decode-heavy serving stays within a few percent of the non-deterministic engine and is often faster, the engine is 2.0–3.7× faster than the only other deterministic engine there, and charges long prefills 12–18% in time to first token, the price of a program that is the same on every GPU.

References

[1] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference." Thinking Machines Lab: Connectionism, 2025.

[2] The SGLang Team, "Towards deterministic inference in SGLang and reproducible RL training." LMSYS Blog, 2025.

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv:2307.08691*, 2023.

[4] Z. Ye *et al.*, "FlashInfer: Efficient and customizable attention engine for LLM inference serving," *arXiv:2501.01005*, 2025.

[5] vLLM contributors, "Bitwise batch invariance for DeepSeek-V4.1-Flash with CUDA graphs." vLLM pull request 62365, 2026.

[6] DeepSeek-AI, "DeepSeek-V4: Towards highly efficient million-token context intelligence," *arXiv:2606.19348*, 2026.

[7] waseemh, "Mlx-batch-invariant: Batch-invariant kernels for MLX on Apple silicon." github.com/waseemh/mlx-batch-invariant, 2025.

[8] llama.cpp contributors, "Deterministic inference mode (CUDA)." llama.cpp pull request 16016 (draft), 2025.

[9] J. Qin, A. Du, D. Zhang, M. Lentz, and D. Zhuo, "Vosti: Specifying, implementing, and verifying deterministic LLM inference," *arXiv:2609.38981*, 2026.

[10] "FoldAttention: Declared-reference softmax for fast decode and deterministic backward," *arXiv:2609.33410*, 2026.

[11] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-decoding for long-context inference." Blog post, 2023.

[12] Z. Yang, N. J. Riasanovsky, W. Deng, and V. Sarkar, "Taming bitwise behavior in GPU kernels with tensor core: Black-box reconstruction, compiler enforcement, and static verification," *arXiv:2609.11356*, 2026.

[13] L. Cooper, S. Jeong, H. Jeon, J. Young, and H. Kim, "Accelerating the mitigation of LLM inference nondeterminism across GPU architectures," *arXiv:2609.25624*, 2026.

[14] Z. Zhang *et al.*, "Deterministic inference across tensor parallel sizes that eliminates training-inference mismatch," *arXiv:2511.17826*, 2025.

[15] "CoRun: Padding is simple and efficient for deterministic LLM inference," *arXiv:2608.14376*, 2026.
[16] "LLM-42: Enabling determinism in LLM inference with verified speculation," *arXiv:2601.17768*, 2026.
[17] J. Demmel and H. D. Nguyen, "Fast reproducible floating-point summation," in *IEEE symposium on computer arithmetic (ARITH)*, 2013.
[18] ROCm contributors, "Stream-K for implicit-GEMM convolution weight gradients." rocm-libraries pull request 13151, 2026.

Appendix A: Additional results

Table 7. Same-GPU baselines, greedy output tokens/s on Qwen3-1.7B (A800). The original and PI rows are a separate launch from Figure 6a, which explains differences of up to 1.8%. vLLM 0.11.2's batch-invariant mode is not batch-invariant on sm_80.

engine	64/256 c1	512/256 c8	512/256 c32	2048/256 c16	4096/128 c4
original	283	1894	5014	1878	539
PI	**298**	**1790**	**4951**	**1863**	**542**
SGLang 0.5.5	253	1768	4865	1837	530
SGLang deterministic	73	554	1993	843	192
vLLM 0.11.2	237	1599	4149	1694	508
vLLM 0.11.2 batch-invariant	244	1539	4083	1671	495
vLLM 0.17.1	270	1842	4890	1885	553
vLLM 0.17.1 batch-invariant	80	609	2228	921	223

Table 8. Long-context, shared-prefix and high-concurrency workloads, PI vs. original (A800; median of three interleaved launches). Throughput is output tokens/s, or requests/s for the 16- and 4-token outputs. The 8B 16K row produces one token per request because 16384 + 256 exceeds the server's context limit.

model	workload (in / out / concurrency)	Δ throughput	Δ TTFT
1.7B	64 / 256 / c1	+5.6%	+8.8%
1.7B	512 / 256 / c32	−0.3%	−0.6%
1.7B	2048 / 16 / c8	−1.7%	+0.1%
1.7B	4096 / 4 / c4	−2.7%	+2.9%
1.7B	4096 / 128 / c8, 3968-token shared prefix	+1.6%	+5.4%
1.7B	8192 / 64 / c4, 8000-token shared prefix	+1.2%	+12.3%
1.7B	16384 / 256 / c2	−3.7%	+12.2%
1.7B	30000 / 128 / c1	−9.4%	+18.3%
8B	64 / 256 / c1	+6.2%	+3.6%
8B	512 / 256 / c32	+1.1%	+3.6%
8B	2048 / 16 / c8	−4.4%	+5.2%
8B	4096 / 4 / c4	−5.8%	+6.3%
8B	4096 / 128 / c8, 3968-token shared prefix	+8.1%	+3.9%
8B	8192 / 64 / c4, 8000-token shared prefix	+5.5%	+6.3%
8B	16384 / 256 / c1 (prefill only)		+13.2%
0.6B	512 / 256 / c64	+1.2%	+6.8%
0.6B	512 / 256 / c128	−2.0%	+18.9%
0.6B	128 / 512 / c128	+1.4%	−2.5%
0.6B	2048 / 128 / c64	−8.0%	+11.3%
1.7B	512 / 256 / c64	+0.7%	+0.8%
1.7B	512 / 256 / c128	−0.3%	+2.3%
1.7B	128 / 512 / c128	+0.9%	−0.1%
1.7B	2048 / 128 / c64	−2.9%	+3.4%
4B	512 / 256 / c64	+6.5%	+2.8%
4B	128 / 512 / c64	+7.5%	+2.8%

Table 9. Batch-1 decode trace by kernel category (Figure 8). Tracing lengthens the gaps between kernels, so these numbers attribute the difference; step times are those of Figure 6a.

per step, PI − original (µs)	0.6B	1.7B	4B	8B
projections	+49	−123	−13	+58
attention	−86	−100	−679	−640
other kernels	−36	+6	−187	−17
between kernels	−73	−46	−324	−303
traced step (original)	2316	3632	7567	11400

Appendix B: Artifact

Code: the engine (`llm_infer_rust`, branch `pi-attn`, kernels in `src/kernels/pi/`) and the research repository (kernels' PyTorch bindings and tests in `kernels/`, detcheck in `detcheck/`, evaluation scripts in `eval/`, Gated DeltaNet in `gdn/`). `eval/run_all.sh` rebuilds the engine for the local GPU and regenerates every result in about 8 hours on one A800; `setup_env.sh` installs every dependency, with exact versions recorded in `requirements/`. `results/README.md` maps every table and figure of this paper to the files that hold its numbers (`results/a800/` for the A800, the top level of `results/` for the RTX 4090 D), and `paper/figures.py` regenerates every figure from them. Exploratory kernel probes, kept and rejected, are documented in `archive/ceiling_probes/`.